

Learnings: een app bouwen met Claude Code

Terugblik op de WWSO Scenario App (19 augustus – 3 oktober 2026), met tips voor het volgende project

Kerncijfer	Waarde	Wat het zegt
Looptijd	6,5 week	Van leeg project naar productie met 3 testers.
Commits	133	Waarvan 27 expliciet bugfixes.
Releases (wijzigingslog)	60	Gemiddeld bijna 10 per week; op 3 oktober alleen al 16 versies.
Drukste dag	23 commits	Veel kleine feedbackrondes achter elkaar.
Testbestanden	45 (318 tests)	Sterk op de rekenmotor, lang zwak op de schermen ertussen.
plan.md + STATUS.md	1.521 regels	Rijk, maar inmiddels te lang om snel in te lezen.
Hetzelfde bugpatroon	8 keer	Staat die tussen pagina's verloren ging of op de verkeerde woning kwam.

De kern in vijf zinnen

- **De basis was sterk:** beleidsboek als bron van waarheid, een losse rekenmotor met golden-master-tests en een wijzigingslog per release. Dat moet je volgende keer precies zo doen.
- **Het zwakke punt zat tussen de schermen**, niet in de berekening: hetzelfde type bug kwam acht keer terug voordat we het structureel oplosten.
- **We bouwden features voor een omgeving die nog niet bestond** (rollen zonder verplichte inlog), waardoor ze in productie het omgekeerde deden van de bedoeling.
- **Feedback kwam in losse druppels en elke druppel werd een release.** Bundelen in rondes met screenshots had veel heen-en-weer gescheeld.
- **Claude Code kon meer zelf bewaken** dan we lieten doen: met een projectinstructie, automatische controles (hooks), een vaste release-stap en voorbeeldomgevingen.

Leeswijzer. Hoofdstuk 1 zegt wat goed ging (behouden). Hoofdstuk 2 bespreekt de tien belangrijkste lessen, telkens met *wat er gebeurde* → *waarom het pijn deed* → *wat je volgende keer doet*. Hoofdstuk 3 laat zien hoe je Claude Code beter inzet. Hoofdstuk 4 is een startchecklist voor het volgende project. Eerlijkheidshalve: een deel van de lessen gaat over keuzes van mij als Claude, niet alleen over jullie aanpak.

1. Wat goed ging — behouden

Wat	Waarom het werkte
Het primaire beleidsboek als bron van waarheid , niet de Excel-interpretatie.	De xlsx bleek op 15 punten af te wijken (o.a. een verschoven rubrieknummering). Door het origineel te volgen kwam dat vroeg boven in plaats van bij de klant.
Rekenmotor als los, puur pakket met golden-master-tests tegen de officiële Huurprijscheck.	Exacte match op elke rubriek. De berekening zelf was de hele periode het meest betrouwbare deel van de app.
Wijzigingslog in de app met een versie per release.	Testers zien wat er veranderd is; jij kunt per versie terugkijken.
Browser-verificatie vóór elke release in plaats van alleen tests.	Vond o.a. de verspringende punten, de overlopende tabel en de uitgerekte schakelaars — dingen die geen unittest vangt.
Echte gebruikers vroeg erbij (Steven, Emma, Myle).	Bijna alle UX-verbeteringen kwamen uit hun feedback, niet uit de specificatie.
Gerichte audits (navigatie, staat-navigatie, voorzieningen).	Elke audit vond bugs die al weken bestonden. Het patroon "eerst kijken, dan bouwen" werkte.
Besluiten vastleggen in STATUS.md, plan.md en het geheugen.	Nieuwe sessies konden verder waar de vorige stopte, ook met een ander model.

2. De tien belangrijkste lessen

Gerangschikt op impact: hoeveel tijd, bugs of verwarring het kostte.

1. Hetzelfde bugpatroon kwam acht keer terug

Wat er gebeurde. Tussen invoer, resultaat en vergelijking werd de "huidige woning" doorgegeven via losse tijdelijke opslagen (sessionStorage), per feature een nieuwe. Telkens raakte een koppeling kwijt of belandde een restje van woning A op woning B. Elke keer werd het symptoom opgelost, pas bij de achtste keer de oorzaak.

Waarom het pijn deed. Elke herhaling kostte een melding, een sessie zoeken, een fix en een release. Eén keer kon het zelfs data overschrijven (woning A opgeslagen met de gegevens van B).

Volgende keer: Bij de **tweede** keer dat een bug op een eerdere lijkt: stop en vraag Claude om een audit van het patroon in plaats van een fix. Schrijf bij elk incident direct een regressietest. En leg architectuurregels vast in de projectinstructie, bijv. "de woning-ID staat altijd in de URL; tijdelijke opslag hoort altijd bij één woning".

2. Features gebouwd voor een omgeving die nog niet bestond

Wat er gebeurde. De rollen en bewerkrechten (eigen org, demo-woning, delen) werden gebouwd terwijl inloggen op productie uit stond om te kunnen testen. Zonder inlog gold elke woning als alleen-lezen, dus elke "Opslaan" maakte een kopie (tot 8 diep) en het automatisch opslaan deed niets.

Waarom het pijn deed. De fout was alleen in productie zichtbaar, niet lokaal en niet in de tests. Het kostte twee releases en het opruimen van 11 kopieën.

Volgende keer: Bouw een feature pas als de omgeving waarin hij moet werken bestaat, of bouw hem bewust "uit" tot die tijd. Vraag Claude bij elke nieuwe feature expliciet: *"hoe gedraagt dit zich met de huidige productie-instellingen?"*

3. Productie was tegelijk de testomgeving

Wat er gebeurde. Testers werkten op productie, met dezelfde database, met inlog uit en de databasebeveiliging open. Testkopieën, testscenario's en echte woningen stonden door elkaar.

Waarom het pijn deed. Elke test van Claude of van testers raakte echte data. Opruimen moest met de hand, en het risico op per ongeluk overschrijven was steeds aanwezig.

Volgende keer: Richt vanaf dag één **twee omgevingen** in: een testomgeving (eigen database, Vercel-preview) en productie. Laat testers op de testomgeving werken en zet inloggen op productie nooit uit.

4. "Gepusht" betekende niet "live"

Wat er gebeurde. Productie werd alleen bijgewerkt met een losse handmatige deploy-opdracht. Na een push stonden fixes dus niet live, wat vandaag tot "ik zie de fixes niet" leidde.

Waarom het pijn deed. Verwarring over wat er live staat, en een tester die een opgeloste bug nog ziet.

Volgende keer: Koppel GitHub aan Vercel, zodat elke push naar de hoofdbranch automatisch deployt en elke branch een eigen voorbeeld-URL krijgt. Of maak een vaste release-opdracht (zie hoofdstuk 3) die altijd eindigt met een controle op de live versie.

5. Feedback in losse druppels, elke druppel een release

Wat er gebeurde. UI-opmerkingen kwamen één voor één binnen, vaak tijdens het bouwen ("nog 1 punt"). Op 3 oktober leidde dat tot 16 versies op één dag.

Waarom het pijn deed. Veel context-wisselingen, veel kleine deploys, en sommige punten raakten elkaar (header, strook en vergelijking) waardoor eerder werk opnieuw moest.

Volgende keer: Verzamel feedback in een **review-ronde**: loop de app door, maak screenshots met een korte notitie per punt, en geef het als één genummerde lijst. Laat Claude daar eerst een plan van maken, en release dan één keer.

6. Grote kant-en-klare agent-opdrachten werkten slechter dan een gesprek

Wat er gebeurde. Twee keer werd een uitgebreide, elders geschreven opdracht geplakt (een test-harness, een volledige navigatie-audit). Beide keren bleek eerst sparren beter: de harness was niet nodig, de audit kon eerst lean.

Waarom het pijn deed. Een vooraf dichtgetimmerde opdracht laat Claude geen ruimte om te zeggen "dit is groter dan nodig" of "dit lost je probleem niet op".

Volgende keer: Begin met het **doel en de context** ("dit gaat mis, zo vaak, dit is de impact") en vraag om een voorstel. Gebruik plan mode voor grotere klussen, zodat je het plan goedkeurt vóór er code geschreven wordt.

7. Technisch correct, maar niet uit te leggen

Wat er gebeurde. De punten achter keukenvoorzieningen werden per vinkje opnieuw uitgerekend en verschoven bij elk ander vinkje. Rekenkundig klopte het, maar niemand kon het uitleggen. Hetzelfde gold voor "type I/II/III" bij parkeren.

Waarom het pijn deed. Gebruikers vertrouwen een getal dat ze niet begrijpen niet, ook als het klopt.

Volgende keer: Laat Claude bij elk scherm met getallen eerst een **mockup** maken (een klikbare pagina of schets) en toets: "*kan ik dit in twee zinnen aan een collega uitleggen?*" Pas bouwen als het antwoord ja is.

8. Een te ruime lezing van het beleid ging ongemerkt mee

Wat er gebeurde. In v0.7.22 concludeerde ik (Claude) dat keuken en sanitair "overal" mochten. Bij de latere audit bleek dat te ruim: buitenruimte, gang en parkeerplek tellen niet. De rekenkern gaf daar ook punten voor.

Waarom het pijn deed. Een interpretatie werd als feit gecommitee, zonder dat iemand hem markeerde als interpretatie.

Volgende keer: Vraag bij elke regel uit het beleid: "*letterlijk of interpretatie? Citeer de paragraaf.*" Laat interpretaties in code en wijzigingslog als zodanig benoemen, en ze apart bevestigen.

9. Projectdocumentatie groeide door tot ruim 1.500 regels

Wat er gebeurde. plan.md en STATUS.md waren het geheugen van het project, maar werden logboeken met elk detail van elke sessie.

Waarom het pijn deed. Elke nieuwe sessie moest veel lezen voordat er gewerkt kon worden, en actuele besluiten raakten verstopt tussen historie.

Volgende keer: Houd drie lagen apart: een korte **CLAUDE.md** (regels en besluiten die altijd gelden), een korte **STATUS** (alleen wat nu speelt) en een **archief** voor sessieverslagen.

10. Afspraken die niet automatisch bewaakt werden, liepen uit de pas

Wat er gebeurde. Voorbeelden: de rekenkern-versie stond twee maanden op 0.0.0 ondanks rekenwijzigingen; Prettier was ingesteld maar bestanden waren niet opgemaakt; het wijzigingslog brak twee keer op een apostrof; wie er testte (Steven, of Steven, Emma en Myle) stond niet eenduidig vast.

Waarom het pijn deed. Elke afspraak die een mens of Claude moet onthouden, gaat een keer mis.

Volgende keer: Laat de omgeving afspraken afdwingen: een **hook** die na elke wijziging opmaakt en typecheckt, een test die faalt als de rekenregels veranderen zonder versieverhoging, en vaste feiten (testteam, omgevingen) in CLAUDE.md in plaats van in losse gesprekken.

3. Hoe je de kracht van Claude Code beter benut

Mogelijkheid	Hoe in te zetten	Wat het hier had opgelost
CLAUDE.md in de projectroot	Doel, bron van waarheid, architectuurregels, omgevingen, testteam en een "definition of done" (tests, typecheck, browser, wijzigingslog, deploy, versiecheck).	Lessen 1, 4, 9 en 10. Nu stond er alleen een Next.js-notitie in apps/web.
Plan mode	Voor elke klus die meer dan een paar bestanden raakt: Claude stelt een plan voor, jij keurt goed, daarna pas code.	Lessen 2 en 6.
Hooks (settings.json)	Automatisch Prettier, ESLint en typecheck na elke bewerking; tests vóór elke commit.	Opmaak- en apostroffouten, vergeten checks.
Eigen slash-commando of skill: /release	Eén stap: versie ophogen, wijzigingslog, tests, commit, push, deploy en live versie controleren.	Les 4 en de handmatige deploy-stappen; ook de geblokkeerde deploy-toestemming.
Toestemmingsregels	Vaste acties zoals deployen naar productie expliciet toestaan of juist altijd laten vragen.	De deploy die door de automatische toestemmingscontrole werd tegengehouden.
/code-review per release	Een tweede blik op de diff voordat hij live gaat; /code-review ultra voor grote wijzigingen.	Les 1 en 8: patronen en te ruime aannames worden eerder gezien.
Subagents voor audits	Parallel laten zoeken (bijv. per scherm of per rubriek) terwijl de hoofdsessie het overzicht houdt.	Snellere audits met minder context in de hoofdsessie.
Artifacts / mockups	Een klikbare pagina met het voorstel vóór er gebouwd wordt.	Les 7: verspringende punten, parkeertypes, header-indeling.
Modelkeuze	Snel model voor kleine UI-klussen; sterk model (Opus) voor architectuur, audits en beleidsinterpretatie.	De staat-navigatie-audit had eerder met een sterk model gekund.
Geheugen (memory)	Korte, blijvende feiten en voorkeuren, zoals "deploy is handmatig" of "testteam = drie personen".	Voorkomt dat dezelfde correctie in een volgende sessie opnieuw nodig is.

Slimmer samenwerken: kleine gewoontes met groot effect

- **Eén bericht, één onderwerp** — of een genummerde lijst. "Nog 1 punt" midden in een klus onderbreekt de lopende verificatie.
- **Screenshot + één zin** werkt beter dan een lange beschrijving: Claude ziet precies wat jij ziet.
- **Zeg wat je wilt bereiken, niet hoe** ("dit moet uit te leggen zijn aan een collega" in plaats van "zet er een tooltip bij").
- **Vraag om een advies met een keuze** wanneer iets een richting is, en om directe uitvoering wanneer het een duidelijke fout is.
- **Gebruik "!"-commando's** voor dingen die jij moet doen (bijv. `! vercel login`); zonder "!" komt het als bericht bij Claude binnen.
- **Laat Claude afsluiten met wat er openstaat**, en begin de volgende sessie met "pak de open punten op".

4. Startchecklist voor het volgende project

Week 0 — vóór de eerste regel code

- Bron van waarheid aanwijzen en in het project zetten (beleid, wet, specificatie) — het origineel, geen afgeleide.
- CLAUDE.md in de root: doel, scope (en wat expliciet *niet*), bron van waarheid, architectuurregels, definition of done.
- Twee omgevingen: test (eigen database + preview-URL) en productie. Inloggen op productie vanaf het begin aan.
- GitHub gekoppeld aan Vercel (automatisch deployen + previews), en een /release-commando voor de vaste stappen.
- Hooks voor opmaak, lint, typecheck en tests.
- Testteam en rollen vastleggen: wie test wat, op welke omgeving.

Tijdens het bouwen

- Grote klus: eerst plan mode, dan bouwen.
- Scherm met getallen of nieuwe interactie: eerst mockup, dan bouwen.
- Elke bug: eerst een test die faalt, dan de fix. Bij de tweede vergelijkbare bug: audit van het patroon.
- Elke beleidsregel: paragraaf citeren, letterlijk of interpretatie benoemen.
- Feedback per ronde verzamelen en in één keer verwerken.

Bij elke release

- /code-review over de wijziging.
- Wijzigingslog in gewone taal, versie omhoog (en rekenkern-versie bij een rekenwijziging).
- Deploy en controle dat de nieuwe versie echt live staat.
- STATUS kort bijwerken: wat is af, wat staat open.

Als je maar vijf dingen onthoudt

#	Les
1	Test- en productieomgeving scheiden vanaf dag één.
2	Bij de tweede herhaling van een bug: oorzaak zoeken, niet symptoom fixen — en altijd een regressietest.
3	CLAUDE.md + hooks + /release: laat de omgeving de afspraken bewaken.
4	Eerst laten zien (plan, mockup), dan bouwen.
5	Feedback bundelen in rondes , met screenshots en één genummerde lijst.